# Valid Parentheses Checker Using Stack

Array-Based and Linked List-Based Interactive Python Implementations

A Beginner-Friendly Textbook Chapter

# Contents

# Chapter 1

# Valid Parentheses Checker Using Stack

**Learning Focus**

**Prerequisites:** None. We begin from first principles. By the end of this chapter, you will be able to design, visualize, implement, debug, and analyze an interactive Python program that asks the user for an expression and checks whether its parentheses, brackets, and braces are balanced using two independent stack implementations.

## Learning Objectives

After studying this chapter, you should be able to:

- Explain why balanced parentheses are important in compilers, editors, data formats, and interviews.
- Identify valid and invalid combinations of `()`, `[]`, and `{}` without immediately writing code.
- Explain why the most recent unmatched opening bracket must be remembered.
- Define and use stack operations: *push*, *pop*, *peek*, *isEmpty*, and *size*.
- Implement a stack using a Python list and using a singly linked list.
- Build an interactive validator using `input()` that ignores non-bracket characters.
- Trace the algorithm using tables, dry runs, memory diagrams, and flowcharts.
- Analyze time and space complexity and discuss improvements and interview follow-ups.

## Motivation, Relevance, and Outcomes

**Why This Matters**

Modern software constantly checks structure. A compiler checks whether `if (x > 0) {` was properly closed. A JSON parser checks whether every object opening brace has a matching closing brace. An IDE highlights syntax errors while you type. The same simple idea appears in expression parsing, template engines, Markdown processors, HTML/XML validation, and technical interviews.

**Expected learning outcome:** You will not merely memorize a solution. You will understand why the stack is the natural tool, how it behaves in memory, how to implement it in two ways, and how to reason about every character processed by the program.

## 1.1 Motivation: Why Validate Parentheses?

Imagine writing this code:

```
if (temperature > 100) {
    print("Warning!");
```

A human notices that the opening brace `{` was never closed. A programming language compiler must also notice this. If it does not, the computer cannot reliably know where the block ends.

Parentheses validation matters in:

- **Compiler syntax checking:** languages such as Python, C, Java, and JavaScript require structural symbols to match.
- **HTML/XML tag matching:** although tags are longer than one character, the idea of nested opening and closing symbols is similar.
- **JSON validation:** arrays use `[]` and objects use `{}`.
- **Mathematical expressions:** `((a+b)*c)` must be structurally meaningful.
- **IDEs and code editors:** editors highlight matching brackets and warn about missing ones.
- **Expression parsing:** parsers often need to know which subexpression is inside which pair of brackets.

**Pause and Predict**

If a program sees `([])`, which opening bracket should match the first closing bracket `}`? The answer is not the first opening bracket. It is the most recent unmatched opening bracket: `{`.

## 1.2 Understanding Balanced Parentheses

For now, do not think about stacks. Think only about structure.

The symbols we care about are:

```
( )    [ ]    { }
```

An expression is balanced when every closing symbol correctly closes the most recent unmatched opening symbol of the same type, and no opening symbol is left unmatched at the end.

### 1.2.1 Valid Examples

| Expression | Why it is valid |
|---|---|
| `()` | The opening parenthesis is closed by the matching closing parenthesis. |
| `(())` | The inner pair closes first; then the outer pair closes. |
| `([])` | The square bracket pair is completely inside the parenthesis pair. |
| `({[]})` | The nesting order is parenthesis, brace, square bracket, then the reverse while closing. |
| `(()())` | Two inner parenthesis groups are correctly inside one outer pair. |
| `([{}])` | Each closing symbol matches the most recent unmatched opening symbol. |

### 1.2.2 Invalid Examples

| Expression | Why it is invalid |
|---|---|
| `(` | An opening parenthesis remains unmatched. |
| `)` | A closing parenthesis appears before any opening parenthesis. |
| `([)]` | The `)` tries to close `[`, but `[` requires `]`. |
| `(]` | The types do not match. |
| `(()` | One opening parenthesis remains unmatched at the end. |
| `}{` | The first symbol is a closing brace with nothing to close. |

**Pause and Predict**

Look at `([)]`. The counts are balanced: two openings and two closings. Is it valid? No. Balanced parentheses require correct *order*, not only equal counts.

## 1.3 Why a Stack?

When scanning left to right, every opening bracket says: "I am waiting to be closed later." If another opening bracket appears before it is closed, the newer bracket must be closed first.

This is the key observation:

> The next closing bracket must match the most recent unmatched opening bracket.

That rule is called **Last In, First Out**, or **LIFO**. The last opening bracket stored is the first one that should be removed when a matching closing bracket appears.

A data structure designed for LIFO behavior is called a **stack**.

## 1.4 Understanding Stack from Scratch

A stack is a collection where insertion and removal happen at one end called the **top**.

### 1.4.1 Analogies

- **Stack of plates:** the last plate placed on top is the first plate removed.
- **Undo operation:** the most recent action is undone first.
- **Browser history:** pressing Back returns to the most recent previous page.
- **Books or pancakes:** you usually remove the top item before reaching lower items.

### 1.4.2 Core Operations

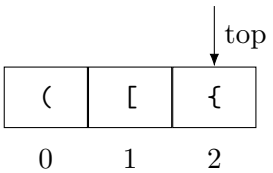
| Operation | Meaning | Example in parentheses validation |
|---|---|---|
| `push(x)` | Put `x` on top | Store an opening bracket such as `(`. |
| `pop()` | Remove and return top item | Remove `(` when a matching `)` appears. |
| `peek()` | Read top item without removing it | Check whether top is `[` before accepting `]`. |
| `is_empty()` | Check whether stack has no items | Detect a closing bracket with nothing to match. |
| `size()` | Count stored items | Useful for debugging and reporting. |

## 1.5 Memory Representation

### 1.5.1 Array Stack

A Python list stores references in contiguous slots. We treat the right end as the top.

```
Top
 v
+-----+ index 2
| { |
+-----+ index 1
| [ |
+-----+ index 0
| ( |
+-----+
```

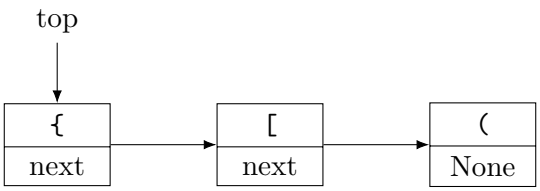



Python lists resize dynamically. When the internal capacity becomes full, Python allocates a larger internal array and copies references. Individual `append()` calls are amortized constant time.

### 1.5.2 Linked List Stack

A linked list stack stores each item in a separate node. Each node contains data and a reference to the next node.

```
Top
 v
+------+------+
| { | *---|----+
+------+------+ |
                v
            +------+------+
            | [ | *---|----+
            +------+------+ |
                           v
                      +------+------+
                      | ( | NULL |
                      +------+------+
```




Linked nodes are dynamically allocated. They do not need contiguous memory, but each node carries overhead because it stores both the value and a reference.

## 1.6 Stack Operations

Each operation below follows the same learning sequence: problem, purpose, visualization, pseudocode, code, output, memory, dry run, and complexity.

### 1.6.1 Push

**Problem:** We need to remember a newly seen opening bracket.
**Purpose:** Add it to the top so it will be matched before older openings.

```
Before push('{'): [(, []
After push('{'): [(, [, {] top = '{'
```

**Algorithm 1** Push Operation

1: **procedure** PUSH(item)
2: place item on top of stack
3: increase size by one
4: **end procedure**

Listing 1.1: Interactive push demonstration using Python list

```python
class ArrayStack:
    def __init__(self):
        self._items = []

    def push(self, item):
        self._items.append(item)

    def __str__(self):
```

```
        return str(self._items)

symbol = input("Enter an opening bracket to push: ")
stack = ArrayStack()
print("Before push:", stack)
stack.push(symbol)
print("After push:", stack)
```

Listing 1.2: Expected console output

```
Enter an opening bracket to push: (
Before push: []
After push: ['(']
```

**Memory before:** empty list. **Memory after:** one list slot references `(`. **Dry run:** input `(`; call `append`; top becomes index 0. **Complexity:** amortized $O(1)$ time, $O(1)$ extra space per push.

### 1.6.2 Pop

**Problem:** A closing bracket arrived, so the most recent opening bracket may now be complete. **Purpose:** Remove and return the top item.

```
Before pop: [(, [, {]
Popped: {
After pop: [(, []
```

**Algorithm 2** Pop Operation

```
procedure POP
    if stack is empty then
        report underflow error
    else
        remove and return top item
    end if
end procedure
```

Listing 1.3: Interactive pop demonstration

```
class ArrayStack:
    def __init__(self):
        self._items = []

    def push(self, item):
        self._items.append(item)

    def pop(self):
        if not self._items:
            raise IndexError("Cannot pop from an empty stack")
        return self._items.pop()

stack = ArrayStack()
expression = input("Enter expression: ")
for character in expression:
    if character in "([{":
        stack.push(character)

```

```
print("Popped opening bracket:", stack.pop())
```

**Expected output:**

```
Enter expression: ([{
Popped opening bracket: {
```

**Complexity:** $O(1)$ time and $O(1)$ extra space.

### 1.6.3 Peek, isEmpty, and Size

**Peek** answers: what is on top? **isEmpty** answers: is there anything to match? **Size** answers: how many unmatched openings remain?

Listing 1.4: Interactive peek, isEmpty, and size demonstration

```
class ArrayStack:
    def __init__(self):
        self._items = []

    def push(self, item):
        self._items.append(item)

    def peek(self):
        if not self._items:
            raise IndexError("Cannot peek at an empty stack")
        return self._items[-1]

    def is_empty(self):
        return len(self._items) == 0

    def size(self):
        return len(self._items)

stack = ArrayStack()
expression = input("Enter expression: ")
for character in expression:
    if character in "([{":
        stack.push(character)

print("Is empty?", stack.is_empty())
print("Size:", stack.size())
if not stack.is_empty():
    print("Top:", stack.peek())
```

## 1.7 Array Stack Implementation

Python's list already supports efficient operations at the right end:

- `append(item)` adds to the top.
- `pop()` removes from the top.
- `items[-1]` reads the top.

Listing 1.5: Production-quality array-based stack

```
class ArrayStack:
    """Stack implementation using Python's built-in list."""
```

```

    def __init__(self):
        self._items = []

    def push(self, item):
        """Place item on top of the stack."""
        self._items.append(item)

    def pop(self):
        """Remove and return the top item."""
        if self.is_empty():
            raise IndexError("Cannot pop from an empty stack")
        return self._items.pop()

    def peek(self):
        """Return the top item without removing it."""
        if self.is_empty():
            raise IndexError("Cannot peek at an empty stack")
        return self._items[-1]

    def is_empty(self):
        """Return True if the stack contains no items."""
        return len(self._items) == 0

    def size(self):
        """Return the number of items in the stack."""
        return len(self._items)

    def snapshot(self):
        """Return a copy for tracing without exposing internal storage."""
        return list(self._items)
```

**Advantages:** simple, fast, compact, built into Python. **Disadvantages:** occasional resizing, contiguous internal storage, less educational about pointers.

## 1.8 Linked List Stack Implementation

A linked list stack needs a `Node` class and a stack class that stores a reference to the top node.

Listing 1.6: Production-quality linked list stack

```
class Node:
    """A single linked-list node."""

    def __init__(self, value, next_node=None):
        self.value = value
        self.next_node = next_node


class LinkedListStack:
    """Stack implementation using a singly linked list."""

    def __init__(self):
        self._top = None
        self._size = 0

    def push(self, item):
        """Place item on top of the stack."""
```

```
        new_node = Node(item, self._top)
        self._top = new_node
        self._size += 1

    def pop(self):
        """Remove and return the top item."""
        if self.is_empty():
            raise IndexError("Cannot pop from an empty stack")
        removed_value = self._top.value
        self._top = self._top.next_node
        self._size -= 1
        return removed_value

    def peek(self):
        """Return the top item without removing it."""
        if self.is_empty():
            raise IndexError("Cannot peek at an empty stack")
        return self._top.value

    def is_empty(self):
        """Return True if there is no top node."""
        return self._top is None

    def size(self):
        """Return the number of nodes."""
        return self._size

    def snapshot(self):
        """Return stack contents from bottom to top for readable tracing."""
        values = []
        current_node = self._top
        while current_node is not None:
            values.append(current_node.value)
            current_node = current_node.next_node
        return list(reversed(values))
```

### 1.8.1 Pointer Movement

During `push('[')`:

```
new_node.value = '['
new_node.next_node = old_top
top = new_node
```

During `pop()`:

```
removed_value = top.value
top = top.next_node
old top node becomes unreachable and may be garbage-collected
```

## 1.9 Problem Statement

Write an interactive program that asks the user to enter an expression and prints whether the expression contains valid parentheses. Only `()`, `[]`, and `{}` participate in validation. All other characters—letters, digits, operators, spaces, quotes, and punctuation—must be ignored.

```
Enter an expression:
([{}])

Output:
The expression contains valid parentheses.
```

```
Enter an expression:
([)]

Output:
The expression contains invalid parentheses.
```

```
Enter an expression:
((a+b)*c)

Output:
The expression contains valid parentheses.
```

```
Enter an expression:
if(x>0){printf("Hi");}

Output:
The expression contains valid parentheses.
```

## 1.10 Designing the Algorithm

### 1.10.1 English Idea

Read the expression from left to right. Store opening brackets. When a closing bracket appears, check whether it matches the most recent stored opening bracket. If it does, remove that opening bracket. If it does not, the expression is invalid. At the end, the expression is valid only if no opening brackets remain.

**Pause and Predict**

What should happen when we encounter a closing bracket but the stack is empty? It must be invalid because there is no earlier opening bracket available to match it.

### 1.10.2 Structured Steps

1. Create an empty stack.
2. Read the expression using `input()`.
3. For each character in the expression:
   (a) If it is an opening bracket, push it.
   (b) If it is a closing bracket, check the stack.
   (c) If the stack is empty, return invalid.
   (d) Otherwise pop the top opening bracket and compare the pair.
   (e) If the pair does not match, return invalid.
   (f) If it is not a bracket, ignore it.

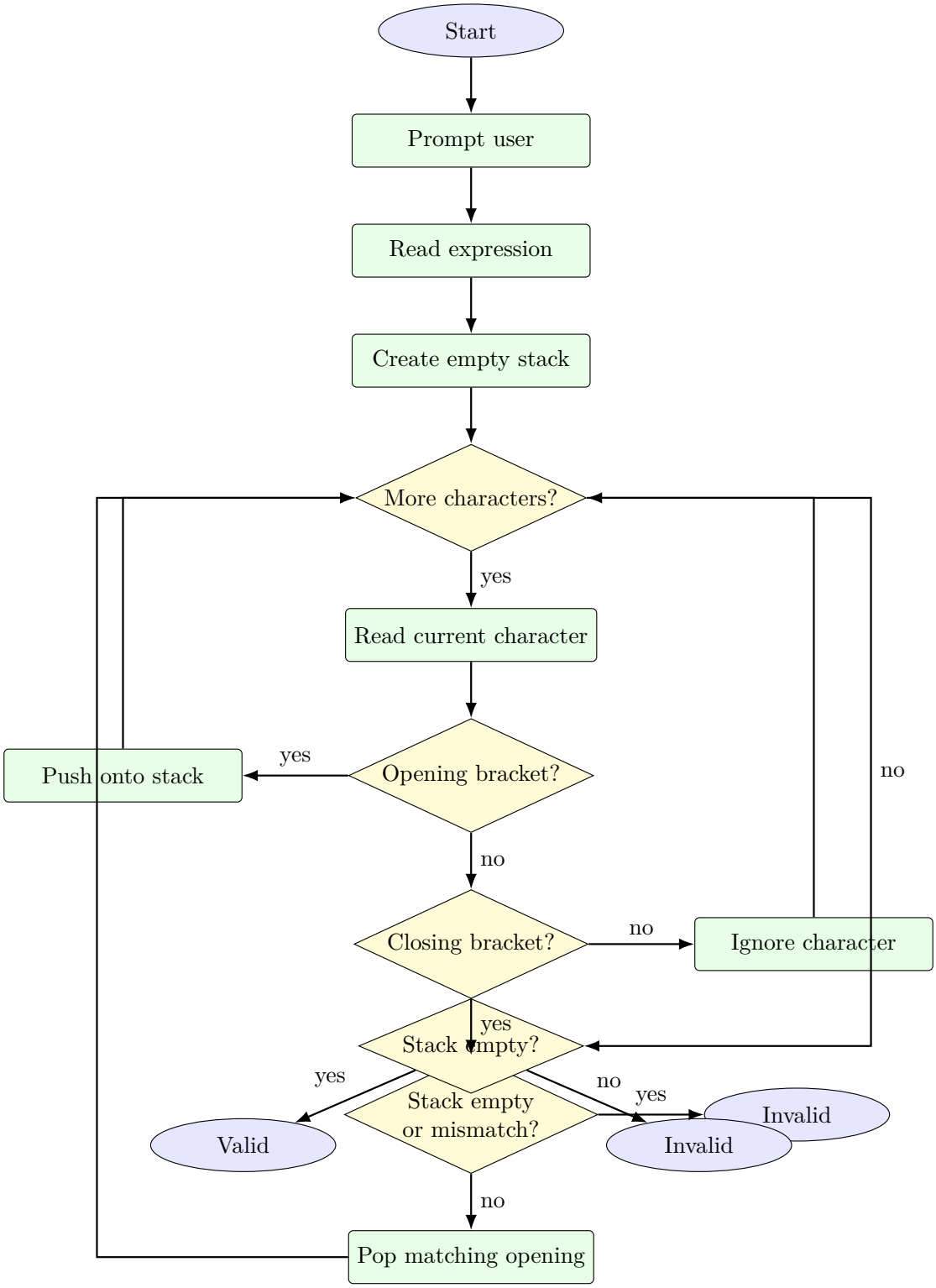
4. After the loop, return valid only if the stack is empty.

**Algorithm 3** Valid Parentheses Algorithm

```
1: procedure IsValidParentheses(expression)
2:     stack ← empty stack
3:     matching ← {) : (, ] : [, } : {}
4:     for each character in expression do
5:         if character is an opening bracket then
6:             push character onto stack
7:         else if character is a closing bracket then
8:             if stack is empty then
9:                 return false
10:            end if
11:            top ← pop stack
12:            if top is not the matching opening bracket then
13:                return false
14:            end if
15:        else
16:            ignore character
17:        end if
18:    end for
19:    return stack is empty
20: end procedure
```

## 1.11 Interactive Program Flow

The program follows one clear path: read the expression, inspect one character at a time, update the stack when necessary, and make a final decision only after all characters have been processed.




**Pause and Predict**

Before looking at the final decision, predict the stack state. If even one opening bracket remains in the stack after the last character, should the expression be valid or invalid? It must be invalid because that opening bracket was never closed.

## 1.12 Visualization Examples

### 1.12.1 Example: `([])`

| Character | Stack Before | Operation | Stack After | Explanation |
|---|---|---|---|---|
| `(` | `[]` | push `(` | `[(]` | opening bracket waits for future match |
| `[` | `[(]` | push `[` | `[(,[]` | nested opening waits |
| `{` | `[(,[]` | push `{` | `[(,[,{]` | most recent opening is brace |
| `}` | `[(,[,{]` | pop `{` | `[(,[]` | brace pair matches |
| `]` | `[(,[]` | pop `[` | `[(]` | square pair matches |
| `)` | `[(]` | pop `(` | `[]` | parenthesis pair matches |
| End | `[]` | final check | `[]` | empty stack means valid |

### 1.12.2 Example: `([)]`

| Character | Stack Before | Operation | Stack After | Explanation |
|---|---|---|---|---|
| `(` | `[]` | push `(` | `[(]` | opening parenthesis waits |
| `[` | `[(]` | push `[` | `[(,[]` | square bracket is now most recent |
| `)` | `[(,[]` | compare with `[` | `[(,[]` | `)` needs `(`, but top is `[`; invalid |

## 1.13 Trace Tables

| `Expression` | Index | Char | Stack Before | Stack After | Operation and Reason |
|---|---|---|---|---|---|
| `()` | 0 | ( | [] | [(] | push opening |
| `()` | 1 | ) | [(] | [] | pop matching (; valid at end |
| `([])` | 0 | ( | [] | [(] | push opening |
| `([])` | 1 | [ | [(] | [(,[] | push opening |
| `([])` | 2 | ] | [(,[] | [(] | pop matching [ |
| `([])` | 3 | ) | [(] | [] | pop matching (; valid |
| `([{}])` | 0 | ( | [] | [(] | push |
| `([{}])` | 1 | [ | [(] | [(,[] | push |
| `([{}])` | 2 | { | [(,[] | [(,[,{] | push |
| `([{}])` | 3 | } | [(,[,{] | [(,[] | pop brace |
| `([{}])` | 4 | ] | [(,[] | [(] | pop square |
| `([{}])` | 5 | ) | [(] | [] | pop parenthesis; valid |
| `(((())))` | 0 | ( | [] | [(] | push |
| `(((())))` | 1 | ( | [(] | [(,(] | push |
| `(((())))` | 2 | ( | [(,(] | [(,(,(] | push |
| `(((())))` | 3 | ) | [(,(,(] | [(,(] | pop |

| Expression | Index | Char | Stack Before | Stack After | Operation and Reason |
|---|---|---|---|---|---|
| ((())) | 4 | ) | [(,(] | [(] | pop |
| ((())) | 5 | ) | [(] | [] | pop; valid |
| ((a+b)*c) | 0 | ( | [] | [(] | push |
| ((a+b)*c) | 1 | ( | [(] | [(,(] | push |
| ((a+b)*c) | 2–4 | a+b | [(,(] | [(,(] | ignore non-brackets |
| ((a+b)*c) | 5 | ) | [(,(] | [(] | pop |
| ((a+b)*c) | 6–7 | *c | [(] | [(] | ignore |
| ((a+b)*c) | 8 | ) | [(] | [] | pop; valid |
| if(a[0]=='(') | 0–1 | if | [] | [] | ignore letters |
| if(a[0]=='(') | 2 | ( | [] | [(] | push |
| if(a[0]=='(') | 4 | [ | [(] | [(,[] | push |
| if(a[0]=='(') | 6 | ] | [(,[] | [(] | pop square |
| if(a[0]=='(') | 9 | ( | [(] | [(,(] | quote ignored, bracket character still counted |
| if(a[0]=='(') | 11 | ) | [(,(] | [(] | pop inner parenthesis |
| if(a[0]=='(') | 12 | ) | [(] | [] | pop outer; valid under simple bracket scanning |
| (a+b)) | 0 | ( | [] | [(] | push |
| (a+b)) | 4 | ) | [(] | [] | pop |
| (a+b)) | 5 | ) | [] | [] | closing with empty stack; invalid |
| ([)] | 0 | ( | [] | [(] | push |
| ([)] | 1 | [ | [(] | [(,[] | push |
| ([)] | 2 | ) | [(,[] | [(,[] | mismatch; invalid |
| { | 0 | { | [] | [{] | leftover opening; invalid at end |
| } | 0 | } | [] | [] | closing with empty stack; invalid |
| ()[] | 0 | ( | [] | [(] | push |
| ()[] | 1 | ( | [(] | [(,(] | push |
| ()[] | 2 | ) | [(,(] | [(] | pop |
| ()[] | 3 | [ | [(] | [(,[] | push |
| ()[] | 4 | ] | [(,[] | [(] | pop square; leftover (; invalid |

## 1.14 Dry Runs

### 1.14.1 Detailed Dry Run for ([])

1. `expression = input(...)` stores the string `([{}])`.
2. `stack = ArrayStack()` creates an empty internal list `[]`.
3. `matching = {')':'(', ']':'[', '}':'{'}` creates the closing-to-opening lookup table.
4. Loop index 0: character `(`. It is opening, so push. Memory: list index 0 stores `(`.
5. Loop index 1: character `[`. It is opening, so push. Memory: index 1 stores `[`; top is index 1.
6. Loop index 2: character `{`. It is opening, so push. Top is now brace.
7. Loop index 3: character `}`. It is closing. Stack is not empty. Pop returns `{`. Compare expected opening `{` with popped `{`; match.

8. Loop index 4: character `]`. Pop returns `[`; match.
9. Loop index 5: character `)`. Pop returns `(`; match.
10. End: stack is empty, so return `True`.

### 1.14.2 Detailed Dry Run for `(a+b))`

At the first `)`, the only stored `(` is popped. At the second `)`, the stack is empty. The program immediately returns `False`. This early return prevents an unsafe pop from an empty stack.

## 1.15 Python Program Using Array Stack

**Complete Array-Based Interactive Solution**

**Problem Statement:** Ask the user for an expression, ignore non-bracket characters, and report whether `()`, `[]`, and `{}` are balanced.
**Intuition:** Opening brackets are promises to close later. The most recent promise must be fulfilled first.
**ASCII Visualization:**

```
Expression: {[(a+b)*x]/y}
Read '{' -> push: [{]
Read '[' -> push: [{, []
Read '(' -> push: [{, [, (]
Read ')' -> pop '('
Read ']' -> pop '['
Read '}' -> pop '{'
End -> stack empty -> valid
```

**Pseudocode:** Use the algorithm already stated in Section 10: push openings, pop and compare closings, ignore all others, final empty-stack check.

Listing 1.7: Array-based valid parentheses checker with runtime input

```python
class ArrayStack:
    """Stack implementation using Python's built-in list."""

    def __init__(self):
        self._items = []

    def push(self, item):
        """Place item on top of the stack."""
        self._items.append(item)

    def pop(self):
        """Remove and return the top item."""
        if self.is_empty():
            raise IndexError("Cannot pop from an empty stack")
        return self._items.pop()

    def is_empty(self):
        """Return True if the stack contains no items."""
        return len(self._items) == 0

    def snapshot(self):
        """Return a safe copy for debugging or tracing."""
        return list(self._items)
```

```python


def has_valid_parentheses(expression):
    """Return True if expression has balanced (), [], and {} symbols."""
    stack = ArrayStack()
    opening_brackets = "([{"
    matching_opening = {
        ")": "(",
        "]": "[",
        "}": "{",
    }

    for character in expression:
        if character in opening_brackets:
            stack.push(character)
        elif character in matching_opening:
            if stack.is_empty():
                return False

            most_recent_opening = stack.pop()
            if most_recent_opening != matching_opening[character]:
                return False

    return stack.is_empty()


def main():
    """Read an expression and print the validation result."""
    expression = input("Enter an expression: ")

    if has_valid_parentheses(expression):
        print("Valid Parentheses")
    else:
        print("Invalid Parentheses")


if __name__ == "__main__":
    main()
```

**Sample Runs:**

```
Enter an expression: (a+b)
Valid Parentheses

Enter an expression: (a+b))
Invalid Parentheses

Enter an expression: {[(a+b)*x]/y}
Valid Parentheses
```

**Output Explanation:** The first and third examples empty the stack exactly at the end. The second example reaches an extra closing parenthesis after the stack has already become empty.

**Memory Diagram:**

```
After reading {[(
index: 0 1 2
```

```
value: { [ (
            top is index 2
```

**Complexity:** If the expression length is $n$, time is $O(n)$. In the worst case, all characters are opening brackets, so stack space is $O(n)$. Best-case invalid input such as `)` returns in $O(1)$.
**Edge Cases:** Empty string is valid because there are no unmatched brackets. Expression without brackets is valid. A single opening or closing bracket is invalid. Letters, numbers, operators, and spaces are ignored.
**Interview Discussion:** The final `return stack.is_empty()` is essential. Without it, `((` would incorrectly appear valid.

## 1.16 Python Program Using Linked List Stack

### Complete Linked List-Based Interactive Solution

**Problem Statement:** Solve the same interactive validation problem, but implement the stack manually using nodes and references.
**Visualization:**

```
After reading "([":
Top -> [ value='[' | next ] -> [ value='(' | None ]
```

**Language-Independent Algorithm:** Create an empty linked stack. Push opening brackets by creating new top nodes. For closing brackets, remove the top node and compare its value with the expected opening bracket.

Listing 1.8: Linked-list-based valid parentheses checker with runtime input

```
class Node:
    """A node in a singly linked list."""

    def __init__(self, value, next_node=None):
        self.value = value
        self.next_node = next_node


class LinkedListStack:
    """Stack implementation using linked nodes."""

    def __init__(self):
        self._top = None
        self._size = 0

    def push(self, item):
        """Place item on top of the stack."""
        new_node = Node(item, self._top)
        self._top = new_node
        self._size += 1

    def pop(self):
        """Remove and return the top item."""
        if self.is_empty():
            raise IndexError("Cannot pop from an empty stack")

```

```
        removed_value = self._top.value
        self._top = self._top.next_node
        self._size -= 1
        return removed_value

    def is_empty(self):
        """Return True if the stack has no nodes."""
        return self._top is None

    def size(self):
        """Return the number of stored items."""
        return self._size

    def snapshot(self):
        """Return values from bottom to top for readable tracing."""
        values = []
        current_node = self._top
        while current_node is not None:
            values.append(current_node.value)
            current_node = current_node.next_node
        return list(reversed(values))


def has_valid_parentheses(expression):
    """Return True if expression has balanced (), [], and {} symbols."""
    stack = LinkedListStack()
    opening_brackets = "([{"
    matching_opening = {
        ")": "(",
        "]": "[",
        "}": "{",
    }

    for character in expression:
        if character in opening_brackets:
            stack.push(character)
        elif character in matching_opening:
            if stack.is_empty():
                return False

            most_recent_opening = stack.pop()
            if most_recent_opening != matching_opening[character]:
                return False

    return stack.is_empty()


def main():
    """Read an expression and print the validation result."""
    expression = input("Enter an expression: ")

    if has_valid_parentheses(expression):
        print("The expression contains valid parentheses.")
    else:
        print("The expression contains invalid parentheses.")

```

```

if __name__ == "__main__":
    main()
```

**Sample Outputs:**

```
Enter an expression: ([{}])
The expression contains valid parentheses.

Enter an expression: ([)]
The expression contains invalid parentheses.
```

**Pointer Movement Dry Run for `([])`:**

1. Read `(`: create node `(`; `top` points to it.
2. Read `[`: create node `[`; its `next_node` points to the `(` node; `top` moves to `[`.
3. Read `]`: pop top node `[`; `top` moves back to `(`.
4. Read `)`: pop top node `(`; `top` becomes `None`; valid.

**Complexity:** Each push and pop is $O(1)$. The full scan is $O(n)$. Space is $O(n)$ in the worst case.
**Advantages:** no resizing copy, explicit dynamic allocation, excellent for learning references. **Disadvantages:** more code and more per-item memory overhead.

## 1.17 Internal Memory Working

### 1.17.1 During Push

Array stack: Python appends a reference at the next available internal slot. If capacity is full, it allocates a larger array and copies references. Linked stack: Python creates a new node object, stores the value, stores old top as `next_node`, then moves `top`.

### 1.17.2 During Pop

Array stack: Python removes the last reference and returns it. Linked stack: the top node's value is saved, `top` moves to the next node, and the old node becomes unreachable. Python's garbage collector may reclaim it later.

### 1.17.3 During Comparison

The dictionary `matching_opening` maps a closing bracket to the only opening bracket that can legally match it. For example, when seeing `]`, the expected popped value is `[`.

### 1.17.4 During Input Processing

`input()` returns one string. The `for character in expression` loop visits each character from left to right. The algorithm does not parse arithmetic or language syntax; it only validates bracket structure.

## 1.18 Complexity Analysis

| Case | Time | Space | Explanation |
|---|---|---|---|
| Best case | $O(1)$ | $O(1)$ | First character is an unmatched closing bracket, such as `)`. |
| Average case | $O(n)$ | up to $O(n)$ | Usually the program scans most or all characters. |
| Worst case | $O(n)$ | $O(n)$ | All characters are openings, such as `((((`. |

| Feature | Array Stack | Linked List Stack | Winner |
|---|---|---|---|
| Implementation length | Short | Longer | Array |
| Push/pop time | Amortized $O(1)$ | Worst-case $O(1)$ | Tie |
| Memory per item | Lower | Higher due to node reference | Array |
| Resizing | Occasional internal resize | None | Linked list |
| Teaching value | Great for Python | Great for pointers | Tie |
| Cache locality | Better | Worse | Array |

## 1.19 Edge Cases

| Input Type | Result | Reason |
|---|---|---|
| empty string | valid | no unmatched bracket exists |
| single opening `(` | invalid | leftover opening remains |
| single closing `)` | invalid | closing appears with empty stack |
| no brackets `abc+12` | valid | all characters are ignored |
| nested `(([]))` | valid | every inner pair closes first |
| mixed `([{}])` | valid | correct reverse closing order |
| wrong mixed `([)]` | invalid | closing type mismatches top |
| long expressions | depends | same rules apply; time grows linearly |
| letters/numbers/operators | ignored | only six bracket symbols participate |
| spaces/special characters | ignored | unless they are bracket symbols |

## 1.20 Debugging

Common bugs include:

- Forgetting the final stack check, causing `((` to be accepted.
- Popping before checking whether the stack is empty.

- Comparing a closing bracket directly to an opening bracket without a mapping.
- Treating equal counts as enough, causing `([)]` to be accepted.
- Accidentally processing non-bracket characters as invalid.

Debugging strategies:

- Print `character` and `stack.snapshot()` at each loop iteration.
- Test one minimal input at a time: `)`, `(`, `()`, `[]`.
- Always ask: "What is the most recent unmatched opening bracket?"

## 1.21 Common Beginner Mistakes

| Incorrect Code | Why It Fails | Correct Code |
|---|---|---|
| `return True` after loop | accepts leftover openings | `return stack.is_empty()` |
| `stack.pop()` before empty check | crashes on `)` | check `is_empty()` first |
| `if ch in all brackets` push | pushes closings too | push only opening brackets: `(`, `[`, `{` |
| `if open == close` | types never equal | use mapping |
| count openings and closings only | accepts `([)]` | use stack order |
| ignore final unmatched openings | accepts `((` | final stack check |
| use `pop(0)` on list | inefficient front removal | use `pop()` |
| use list beginning as top | slower shifts | use right end |
| forget quotes in dictionary keys | syntax error | use quoted dictionary keys |
| use `=` in condition | syntax error | use `==` |
| use `is` for string equality | unreliable style | use `==` |
| miss `{}` pair | incomplete validation | include all three pairs |
| return invalid for letters | violates requirement | ignore non-brackets |
| forget `self` in methods | method error | include `self` |
| expose internal list | unsafe design | provide `snapshot()` |
| forget size decrement | wrong linked size | decrement after pop |
| lose linked next pointer | breaks chain | set `new_node.next_node = top` |
| not moving top on pop | infinite stale top | `top = top.next_node` |
| compare after popping empty | exception | guard first |
| use mutable class variable list | shared stacks | create list in `__init__` |

## 1.22 Interview Insights

Interviewers like this problem because it tests fundamentals: scanning, conditionals, maps/dictionaries, stack behavior, edge cases, and proof of correctness.

Common questions:

- Why is a stack preferred over a queue? Because we need most recent unmatched opening first.
- Can we solve it with counts? Not correctly for mixed nesting.
- What are follow-ups? HTML tag validation, longest valid parentheses, minimum removals, and streaming validation.
- Can we optimize space? Worst-case $O(n)$ is necessary for arbitrary nesting depth.

## 1.23 Applications

The same idea appears in compiler parsing, IDE syntax checking, expression evaluation, JSON validation, HTML parsing, Markdown parsing, programming language interpreters, template engines, XML validation, and source-code linters.

## 1.24 Variations

- **Minimum Parentheses Removal:** remove the fewest parentheses to make a string valid.
- **Longest Valid Parentheses:** find the longest valid parenthesis substring.
- **Generate Parentheses:** generate all valid combinations for $n$ pairs.
- **HTML Tag Matching:** match multi-character opening and closing tags.
- **Balanced Symbols:** extend to quotes or custom delimiters.
- **Expression Conversion:** use stacks for infix, postfix, and prefix conversion.
- **Postfix Validation:** verify that an expression has enough operands for operators.

## 1.25 FAQs

1. **What is the main goal?** To verify that every opening bracket is closed by the correct closing bracket in the correct order.
2. **Why not just count brackets?** Counts miss order errors like `([)]`.
3. **What does LIFO mean?** Last In, First Out.
4. **Why does LIFO fit?** The most recent opening bracket must close first.
5. **What happens for an empty expression?** It is valid because nothing is unmatched.
6. **Are letters invalid?** No. They are ignored.
7. **Are quotes handled specially?** In this simple validator, no; bracket characters inside quotes are still scanned.
8. **How would we ignore brackets inside strings?** We would add lexical analysis or state tracking for quotes.

9. **What is stack underflow?** Trying to pop from an empty stack.
10. **What is the top of stack?** The item most recently pushed.
11. **Why use a dictionary?** It gives clear closing-to-opening matching logic.
12. **What is the time complexity?** $O(n)$.
13. **What is the space complexity?** Worst-case $O(n)$.
14. **Which stack implementation is better in Python?** Usually the list-based stack.
15. **Why learn linked lists then?** They teach dynamic memory and references.
16. **Can this process a file?** Yes, read the file content as a string and pass it to the function.
17. **Does it validate full Python syntax?** No, only bracket balance.
18. **Can it validate HTML?** Only with extensions for tags.
19. **Why final empty check?** It catches unmatched openings.
20. **When can we return early?** On empty-stack closing or mismatched pair.
21. **Is `abc` valid?** Yes, no bracket rules are violated.
22. **Is `(abc` valid?** No, `(` remains unmatched.
23. **What data structure would be wrong?** A queue, because it removes oldest first.
24. **Can recursion solve it?** Sometimes, but a stack is clearer and iterative.
25. **What is amortized $O(1)$?** Average constant time over many list appends despite occasional resizing.
26. **Can the linked list stack overflow?** It is limited by available memory.
27. **Should pop return a value?** Yes, we need to compare it.
28. **Should peek be used instead of pop?** Pop is natural because a matched opening is no longer unmatched.
29. **Can Unicode brackets be added?** Yes, extend the mapping.
30. **What is the biggest beginner insight?** Correct nesting order matters more than counts.

## 1.26 Cheat Sheet

**Valid Parentheses Cheat Sheet**

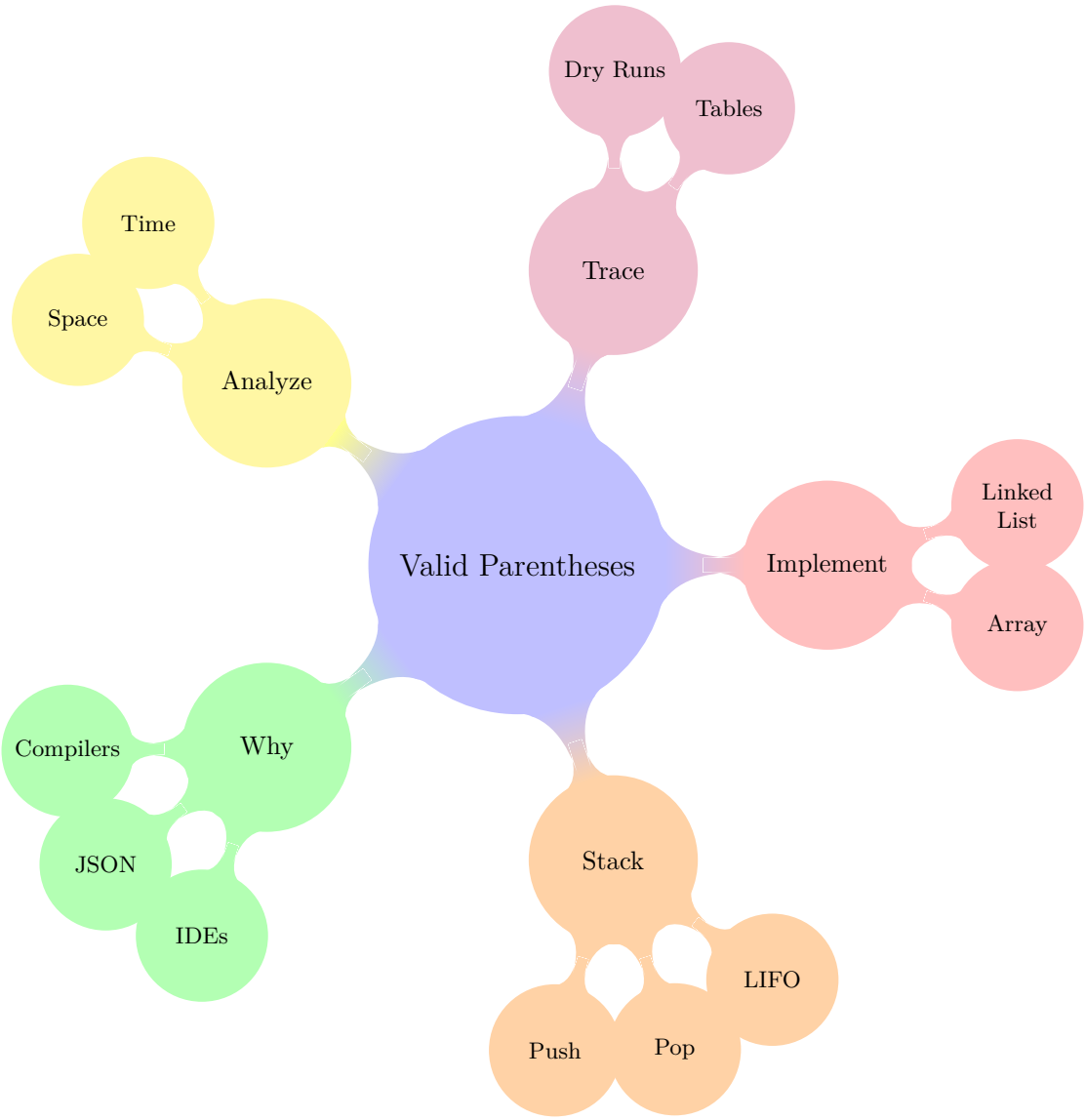
- Opening bracket $\rightarrow$ push.
- Closing bracket $\rightarrow$ stack must not be empty.
- Closing bracket $\rightarrow$ popped opening must match.
- Non-bracket character $\rightarrow$ ignore.
- End of input $\rightarrow$ stack must be empty.
- Time $O(n)$, space $O(n)$.
- Main pitfall: forgetting unmatched openings at the end.

## 1.27 Mind Map

### 1.27.1 ASCII Mind Map

```
Valid Parentheses
|-- Why
| |-- compilers
| |-- JSON
| `-- IDEs
|-- Stack
| |-- push openings
| |-- pop on closings
| `-- LIFO
|-- Implementations
| |-- Python list
| `-- linked list nodes
|-- Algorithm
| |-- scan left to right
| |-- ignore non-brackets
| `-- final empty check
`-- Analysis
    |-- O(n) time
    `-- O(n) space
```

### 1.27.2 TikZ Mind Map




## 1.28 Revision Notes

- Think: "closing bracket checks the most recent opening bracket."
- If the stack is empty and a closing bracket arrives, invalid immediately.
- If the loop ends and the stack is not empty, invalid.
- Python list stack: use `append()` and `pop()` at the right end.
- Linked list stack: top pointer moves during every push and pop.
- Interview memory trick: open means "store", close means "verify and remove".

## 1.29 Practice Questions

### Easy

1. Trace `()`.

2. Trace `[]{}`.
3. Decide whether `(]` is valid.
4. Decide whether `abc` is valid.
5. Explain why `)` is invalid.
6. Write the dictionary for matching brackets.
7. Explain push using a plate analogy.
8. Explain pop using browser history.
9. What is the final stack for `((`?
10. Why are letters ignored?

### Medium

11. Implement `peek()` for `ArrayStack`.
12. Implement `size()` for `LinkedListStack`.
13. Add Unicode bracket support.
14. Print a trace table while validating.
15. Return the index of the first error.
16. Count maximum nesting depth.
17. Compare `([)]` and `([])`.
18. Modify output to say which bracket is missing.
19. Validate multiple lines until the user types `quit`.
20. Write unit tests for ten examples.

### Hard

21. Ignore brackets inside quoted strings.
22. Validate HTML-like tags.
23. Find minimum removals to make parentheses valid.
24. Find longest valid parentheses substring.
25. Build a GUI bracket checker.
26. Stream characters one at a time from a file.
27. Explain proof of correctness using loop invariants.
28. Convert infix to postfix using a stack.
29. Validate JSON brackets and quote states.
30. Build a source-code syntax checker prototype.

## 1.30 Coding Exercises

1. Fill in the missing `push()` method for an array stack.
2. Fill in the missing `pop()` method for a linked list stack.
3. Write a trace-printing version of `has_valid_parentheses()`.
4. Replace the dictionary with a helper function `matches(opening, closing)`.
5. Add a menu allowing the user to choose array stack or linked list stack.
6. Draw the stack after every character in `{[(x)]}`.
7. Write a program that reports maximum nesting depth.
8. Write a bug-fixing exercise for `return True` after the loop.
9. Create a fill-in-the-blank linked node constructor.
10. Build a visualization that prints `PUSH`, `POP`, and `IGNORE`.

## 1.31 Debugging Exercises

### 1.31.1 Broken Implementation 1

```
def broken_validator(expression):
    stack = []
    for character in expression:
        if character in "([{" :
            stack.append(character)
        elif character in ")]}":
            stack.pop()
    return True
```

**Bugs:** It pops without checking empty stack, never checks matching type, and always returns `True`. **Fix:** add empty check, mapping comparison, and final empty check.

### 1.31.2 Broken Implementation 2

```
def broken_validator(expression):
    opening_count = 0
    closing_count = 0
    for character in expression:
        if character in "([{" :
            opening_count += 1
        elif character in ")]}":
            closing_count += 1
    return opening_count == closing_count
```

**Bug:** Equal counts do not prove correct order or type. `([)]` has equal counts but is invalid. **Fix:** use a stack.

### 1.31.3 Broken Implementation 3

```
def broken_validator(expression):
    stack = []
    matching_opening = {")": "(", "]": "[", "}": "{"}
```

```
    for character in expression:
        if character in "([{" :
            stack.append(character)
        elif character in matching_opening:
            if stack[-1] != matching_opening[character]:
                return False
            stack.pop()
    return len(stack) == 0
```

**Bug:** `stack[-1]` fails if a closing bracket appears while the stack is empty. **Fix:** check `if not stack:  return False` first.

## 1.32 Mini Projects

- **Interactive Parentheses Validator:** ask the user for repeated expressions and show a trace table for each one.
- **HTML Tag Validator:** push opening tags such as `<div>` and pop when `</div>` appears.
- **Simple Expression Checker:** validate brackets and report maximum nesting depth.
- **JSON Bracket Validator:** validate `[]` and `{}` while tracking quoted strings.
- **Source Code Syntax Checker:** read a source file and report line and column of the first bracket mismatch.

## Final Encouragement

A stack is powerful because it matches how nested structure works. Whenever a problem says "the most recent unfinished thing must be completed first," pause and ask: "Is this a stack problem?" For valid parentheses, the answer is yes.